Crystal — Vibecon Case Study

Submission for: Vibecon Judge Panel **Project**: Crystal (published to PyPI as `crystal-code`) **Website**: crystalcodes.dev **License**: MIT · Open source **Built with**: Emergent (full-stack, end-to-end)

---

1. The One-Line Pitch

Crystal is the first code-quality tool built specifically for vibe coders — the people shipping products with Cursor, Claude, Windsurf, Bolt, Replit, and Emergent. It watches the project across AI sessions, catches the failures AI tools reliably produce, and generates paste-ready handoffs and fix prompts in plain English.

---

2. The Problem (who it's for, why it matters)

The persona

A solo founder or 2–3 person team shipping a real product using AI coding tools. Not a FAANG engineer. Not an enterprise CTO. Someone who writes prompts, not code — or who writes some code and lets AI fill in the rest.

The pain, in concrete terms

Every vibe coder hits the same four walls between session 1 and session 10:

1. **Context loss** — every new AI chat starts from zero. You retype the same 3-sentence "we're building a React + FastAPI + MongoDB todo app, the auth uses…" preamble. Your AI forgets the architectural decisions you made 2 sessions ago.
2. **Silent drift** — an API key slides into `src/config.js`. A `MongoClient` import appears in `App.jsx`. A test file gets deleted because the AI couldn't fix it. You don't notice until deployment.
3. **Unsafe defaults** — `allow_origins=["*"]` in FastAPI. `.env` not in `.gitignore`. `http://localhost:8000` hardcoded in 6 files. Works on your laptop, breaks on staging.
4. **No handoff** — you switch from Cursor to Claude mid-feature. The second tool knows nothing about what the first tool did. You lose 20 minutes re-orienting.

Market-level evidence

- **36%** of developers using AI to code never run tests on the output (Stack Overflow 2024 Developer Survey)
- **1.7×** more bugs in AI-generated code vs human code (GitClear, *Copilot Impact on Code Quality*, 2024)
- **45%** of AI-generated code contains exploitable security flaws (Stanford / NYU study, *Do Users Write More Insecure Code with AI Assistants?*)
- The AI coding tool market is estimated at ~10M+ active users across Cursor, Claude, Replit, Bolt, Windsurf, Emergent (compounded market figures from each vendor's 2024 disclosures). Growing ~15% QoQ.

The gap: Linters watch files. CI pipelines watch builds. Type-checkers watch types. **Nothing watches the project across AI sessions.** That's Crystal's slot.

---

3. The Insight (what others miss)

> *The bug isn't in any single file.* ***It's between sessions.***

Every existing code-quality tool assumes a stable team of humans with shared context. When the team is *you + rotating AI assistants with no memory*, the failure mode is completely different:

- The AI doesn't forget *syntax* — it forgets *decisions*.
- The AI doesn't write *wrong* code — it writes *locally correct, globally wrong* code (a DB query that's fine, but in the wrong layer).
- The AI doesn't miss *tests* — it *silently deletes them* when it can't make them pass.

Crystal is built around this observation. It doesn't try to replace lint, CI, or type-checking. It watches the three things those tools can't see: **session boundaries, domain boundaries, and project-level integrity over time.**

---

4. The Product (what we shipped)

Real, installable, tested

```
pip install crystal-code
```

- [redacted] Crystal in ~10 seconds.
- **Live landing page**: crystalcodes.dev
- **161 automated tests**, **82% coverage** across 16 test files. Run `pytest --cov=src/crystal_guard` to verify.
- **Zero open CRITICAL or HIGH bugs** in the codebase (run `crystal check` on itself — it passes its own gates).

The 16 commands (all ship in v0.3.0)

Command	What it does
`crystal init`	Auto-detects stack (React, Next.js, Vue, Django, Python, MongoDB, Postgres, etc.) and creates `.crystal/` config.
`crystal check`	Runs all gates. `--stage local
`crystal audit`	Deep audit with hotspot correlation. `--llm` adds natural-language senior-engineer review.
`crystal fix`	Safe, whitelisted auto-fixes. Never touches source code. Dry-run by default.
`crystal fix-prompt`	Generates paste-ready AI prompts for every issue, with WHY explanations.
`crystal diff`	Checks only files changed since the last git commit.
`crystal handoff`	One paste. Your next AI session knows everything.
`crystal completeness`	Compares PRD checklist with actual implementation.
`crystal test`	Runs real tests (pytest / jest / vitest auto-detected).
`crystal status`	Score + trends + recent changes.
`crystal gates`	Shows each quality gate individually.

`crystal report`	Detailed markdown report for PRs.
`crystal architect`	Generates `architecture.md` with enforced rules.
`crystal badge`	Shields.io-compatible health badge.
`crystal rules`	Manage architecture rules from CLI.
`crystal mcp serve`	Runs the MCP server for Cursor / Claude / Windsurf.

The 20 quality gates

Architecture (4) — expected directories exist, required files present, config sprawl, deep nesting **Domain purity (3)** — frontend purity, backend purity, cross-layer env vars **Security (4)** — hardcoded keys, passwords, recognized key formats (Stripe/OpenAI/GitHub/AWS), .env-in-.gitignore **Code hygiene (4)** — TODO/FIXME, placeholders (example.com), debug logging, hardcoded localhost **Dependencies (3)** — pip-audit / npm-audit vulnerabilities, unused packages, duplicate functionality (axios + node-fetch) **Stage-specific (2)** — no localhost URLs (staging+), all env vars defined (staging+)

Architecture of Crystal itself

```
crystal-guard/
├── src/crystal_guard/
|   ├── cli.py          # 16 Typer commands
|   ├── agent.py        # AI audit / completeness / refactor
|   ├── pipeline.py     # stage-aware checking
|   ├── handoff.py      # session handoff generator
|   ├── fixers.py       # per-rule auto-fixers (whitelist)
|   ├── badge.py        # shields.io-compatible badge
|   ├── analyzers/      # 5 analyzers
|   ├── rules/builtin/  # 5 stack rule packs (YAML)
|   ├── scoring/        # A-F grade + weighted severity
|   └── mcp/            # FastMCP server: tools, resources, prompts
└── tests/              # 161 tests, 82% coverage
```

5. Proof: Real Before → After (reproducible)

The repo ships a case-study project at `examples/case-study/` — a real React + FastAPI + MongoDB todo app with all the classic vibe-coded problems.

Anyone can reproduce these numbers in 30 seconds:

```
git clone https://github.com/ashimnandi-trika/crystal
cd crystal/project-crystal/crystal-guard
pip install -e .
crystal check examples/case-study/before
crystal check examples/case-study/after
```

Before

Metric	Value
Grade	**F**
Score	**0** / **100**
Critical issues	5
High issues	3
Medium issues	7
Low issues	14
Total	**29 issues**
Stage gate	**BLOCKED** (cannot ship)

Sample criticals caught:

- `[CRITICAL] sec-001` — Hardcoded OpenAI API key in `frontend/src/config.js:15`
- `[CRITICAL] sec-003` — Stripe live key format in `backend/server.py`
- `[CRITICAL] dom-001` — `MongoClient` import in `App.jsx` (database code in browser)

- [redacted]
- `[HIGH] sec-005` — CORS wildcard `allow_origins=["*"]`

After (~2 hours of work using `crystal fix-prompt`)

Metric	Value
Grade	**A**
Score	**98** / **100**
Critical issues	**0**
High issues	**0**
Total	**1** (a `LOW` missing README)
Stage gate	**PASSED** (ready to ship)

Takeaway

- **29 → 1 issue** with zero infrastructure changes.
- **5 critical security/domain violations → 0.**
- All fixes generated as **paste-ready AI prompts** with WHY explanations. No memorizing rules. No reading docs. Just paste into Cursor/Claude and ship.

---

6. Originality (what makes Crystal different)

1. **Session-aware, not file-aware.** Crystal is the only tool that treats the AI-coding session as the unit of integrity. Its `handoff`, `diff`, `audit`, and `completeness` commands exist because no linter understands that "session" is a first-class concept.
2. **Plain-English output.** Every rule has a WHY explanation written for someone who has never coded. "Your API key is like a credit card number. Right now it's written directly in your code…" — not "sec-001 violation."
3. **Stage-aware strictness.** Most tools are binary (fail or pass). Crystal models `local → staging → production` with increasing strictness. Your `TODO` is fine at local stage, blocks at production.
4. **MCP-native.** Crystal isn't a silo. It plugs directly into Cursor, Claude Desktop, and Windsurf via Model Context Protocol. The AI sees the gates in real time.

5. **Every feature works offline rule-based.** `--llm` is opt-in, and supports Anthropic, OpenAI, and the Emergent universal key. No vendor lock.
6. **Whitelist auto-fix.** `crystal fix` only fixes safe, filesystem-level operations (add .env to .gitignore, create tests/ scaffold). It never, ever modifies source code. This is the difference between a tool you trust and one you don't.

---

7. How Emergent was used

Crystal is built entirely on Emergent, end-to-end.

- **Landing page** (`crystalcodes.dev`) — React SPA, custom components, full SEO, OG social card, all shipped through Emergent.
- **Backend API** — FastAPI server with MongoDB, pagination, CORS, health endpoint — all scaffolded and hardened through Emergent's agent.
- **Python CLI package** — 16 commands, 5 analyzers, stage pipeline, MCP server, 161 tests, published to PyPI — built through iterative Emergent sessions.
- **Dogfooding**: Crystal's own `crystal audit --llm` feature uses the **Emergent universal LLM key** (`emergentintegrations`) as one of its supported providers. Crystal literally uses Emergent's AI substrate to analyze AI-generated code.
- **Session handoff validation**: Crystal's handoff generator was tested across real Emergent fork sessions. This case study's fixture project was refactored using the same `crystal handoff` prompt pasted between sessions — the tool validates itself.

Emergent's value here: **the ability to iterate a full-stack product (landing + backend + CLI + docs) with a single agent loop** meant Crystal went from idea to PyPI in weeks, not months.

---

8. Validation signals (as of submission)

- ✓ **Published on PyPI**: `pip install crystal-code` (v0.3.0 ready, v0.2.0 live). Real install count growing.
- ✓ **Live production site**: crystalcodes.dev deployed via Emergent + Cloudflare.
- ✓ **Open source, MIT**: full codebase on GitHub. Reproducible case study shipped in `examples/`.
- ✓ **MCP integration**: verified to work with Cursor, Claude Desktop, Windsurf.
- ✓ **Zero dogfood bugs**: Crystal passes its own production-stage check.

---

9. Market potential & path

TAM

~10M+ developers use AI coding tools today. Each is a latent Crystal user.

Path to 100 users (next 30 days)

- PyPI downloads + HackerNews launch (`Show HN: Crystal — clean code that ships`)
- 5 stack rule packs ship v0.3.0 (React+Python, Next.js+Prisma, Vue+Node, Django, generic) — each covers a well-defined community
- MCP marketplace listing on Cursor + Claude Desktop
- Free forever; no signup required for CLI

Path to first revenue (next 6 months)

- **Crystal Teams** ($19/mo): shared rules across repos, PR-level dashboard, cross-session history, team-wide debt tracking
- **Crystal Cloud** ($99/mo, later): hosted check API, GitHub PR bot, Slack alerts, audit trail for compliance
- CLI stays free forever. Monetization is for teams, not individuals.

Why now

AI coding adoption is hitting the "it works but nobody trusts it" phase. The next 12 months belong to tools that add trust back. Crystal is that tool for the individual vibe coder — the exact audience Vibecon convenes.

---

10. What we're asking judges for

Specific, falsifiable feedback on two open trade-offs:

1. **Gate strictness defaults** — is `production` too harsh? Too soft? At what point do vibe coders turn it off?
2. **LLM-layer cost vs. depth** — should `audit --llm` be the default? Does the rule-based layer alone close enough of the quality gap?

General "this is cool" is appreciated. Specific takes on either question would help Crystal more.

11. Team–Product fit

Ashim Nandi — built Crystal because *I am Crystal's first user*.

I've shipped products on Cursor, Claude, and Emergent. Every one of the failure modes Crystal catches has personally burned me:

- I've committed `.env` to GitHub. Had to rotate keys at 2am.
- I've had an AI delete my test file to "fix the build." Didn't notice for a week.
- I've lost 40 minutes re-orienting a fresh Claude session because I didn't write a handoff.

Crystal isn't theoretical. Every rule, every WHY explanation, every command exists because I hit that specific wall. That's the team-product fit: the person writing the tool is the target user, 30 days ago.

---

12. Links & commands

- **Install**: `pip install crystal-code`
- **Website**: crystalcodes.dev
- **Source**: github.com/ashimnandi-trika/crystal
- **Case study repo path**: `project-crystal/crystal-guard/examples/case-study/`
- **License**: MIT
- **Version**: 0.3.0

Judges: reproduce the headline numbers in 30 seconds

```
pip install crystal-code
git clone https://github.com/ashimnandi-trika/crystal
cd crystal/project-crystal/crystal-guard
crystal check examples/case-study/before   # F, 0/100, 29 issues
crystal check examples/case-study/after    # A, 98/100, 1 issue
crystal audit examples/case-study/before --llm   # senior-engineer review
crystal handoff examples/case-study/after        # the handoff prompt itsel
```

Crystal. Clean code that ships.